

IAM TASK

1. Create one IAM user and assign EC2 and S3 full access roles.

Step-1: AWS Console → IAM

Step-2: Create user → IAM → Users

Step-3: Set Permissions → Choose: Attach policies directly

Step-4: Search for: AmazonEC2FullAccess

Verify

Step-5: Result: AdminUser | EC2 Full Access | S3 Full Access

IAM

>

IAM users

>

Create user

Step 1
Specify user details

Step 2
Set permissions

Step 3
Review and create

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐ Add user to group
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions
Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒ Attach policies directly
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1/1571)

Choose one or more policies to attach to your new user.

Q AmazonEC2FullAccess

Filter by Type
All types

1 match

☒	Policy name	Type	Attached entities
☒	AmazonEC2FullAccess	AWS managed	0

[Create policy](#)

► Set permissions boundary - optional

Review and create

Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.

User details

User name	Console password type	Require password reset
AdminUser	None	No

Permissions summary

Name	Type	Used as
AmazonEC2FullAccess	AWS managed	Permissions policy

Tags - optional

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tags.

IAM TASK

Load the user's password and email instructions for signing in to the AWS Management Console.

Retrieve password

You can view and download the user's password below or email users instructions for signing in to the AWS Management Console. This is the only time you can view and download this password.

Console sign-in details

Email sign-in instructions

Console sign-in URL
https://567487919764.signin.aws.amazon.com/console

User name
Sunith

Console password
Show

Cancel

Download .csv file

Return to users list

Account ID: 5674-8791-9764

Sunith

Account ID
5674-8791-9764

Account name
Access denied

Account colour
Access denied

IAM user
Sunith

Account

Organisation

Service Quotas

Billing and Cost Management

Security credentials

Console mobile app

Agent Toolkit for AWS

Turn on multi-session support

IAM users (1/1) Info

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

Search

☒	User name	Path	Group	Last activity	MFA	Password age	Console last sign-in
☒	Sunith	/	0	4 minutes ago	-	2 minutes	4 minutes ago

IAM TASK

2. Create one group in IAM and assign read access for EC2.

Step-1:Open Groups

IAM → User groups → Create group

Group Name → EC2-ReadOnly-Group

Step-2:Attach Policy : AmazonEC2ReadOnlyAccess

Step-3:Verify: IAM → User groups → EC2-ReadOnly-Group → Permissions

What can this group do?

EC2 instances

AMI information

Security groups

Volumes

Snapshots

Networking information

IAM user groups (1) [Info](#)

A user group is a collection of IAM users. Use groups to specify permissions for a collection of users.

☐	Group name	Users	Permissions	Creation time
☐	EC2-ReadOnly-Group	0	Defined	Now

EC2-ReadOnly-Group user group created.

[View group](#)

EC2-ReadOnly-Group [Info](#) [Delete](#)

Summary [Edit](#)

User group name
EC2-ReadOnly-Group

Creation time
September 02, 2026, 15:00 (UTC+05:30)

ARN
[arn:aws:iam::567487919764:group/EC2-ReadOnly-Group](#)

Users

Permissions

Access Advisor

Permissions policies (1) [Info](#)

You can attach up to 10 managed policies.

Filter by Type [All types](#) [Simulate](#) [Remove](#) [Add permissions](#)

☐	Policy name	Type	Attached entities
☐	AmazonEC2ReadOnlyAccess	AWS managed	1

IAM TASK

3. Create a new user named "Devops" and add to the group created in task 2.

Step-1: IAM → Users → Create User = Devops user

Step-2: Permissions: Choose → Add user to group → EC2-ReadOnly-Group

Step-3: Verify : IAM → Users → Devops → EC2-ReadOnly-Group → Users

The user gets permissions from the **group**.

Verify: Devops → Group → EC2 ReadOnly Policy → EC2 Read Permissions

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☒ Add user to group

Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions

Copy all group memberships, attached managed policies, and inline policies from an existing user.

☐ Attach policies directly

Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

User groups (1/1)

Search



Create group

< 1 > ⚙

☒	Group name ↗	Users	Attached policies ↗	Created
☒	EC2-ReadOnly-Group	0	AmazonEC2ReadOnlyAccess	2026-09-02 (7 minutes ago)

► Set permissions boundary - optional

Devops [Info](#)

Delete

Summary

ARN
 arn:aws:iam::567487919764:user/Devops

Console access
 Enabled without MFA

Access key 1
[Create access key](#)

Created
September 02, 2026, 15:08 (UTC+05:30)

Last console sign-in
 Never

Permissions [Groups \(1\)](#) Tags Security credentials Last Accessed

User groups membership

Remove

Add user to groups

A user group is a collection of IAM users. Use groups to specify permissions for a collection of users. A user can be a member of up to 10 groups at a time.

☐	Group name	Attached policies ↗
☐	EC2-ReadOnly-Group	AmazonEC2ReadOnlyAccess

IAM TASK

4. Write a bash script to create an IAM user with VPC full access.

Steps-1:create:

Create → User: VPCAdmin

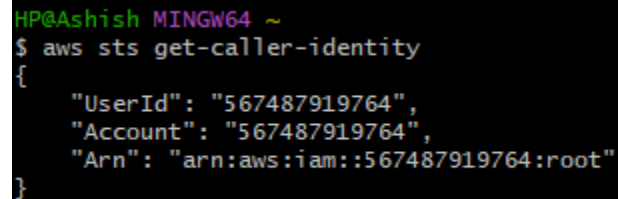
Attach: AmazonVPCFullAccess

Step-2:Check AWS CLI

aws - -version

Step-3:Check AWS Identity

aws sts get-caller-identity



```
HP@Ashish MINGW64 ~
$ aws sts get-caller-identity
{
  "UserId": "567487919764",
  "Account": "567487919764",
  "Arn": "arn:aws:iam::567487919764:root"
}
```

Step-4:Check Your Region

aws configure get region

Step-5:If you want your lab to use US East 1:

aws configure set region us-east-1

Step-6:Check:

aws configure get region

Step-7:Create Script:

vim create-vpc-user.sh

```
#!/bin/bash
```

```
USER_NAME="VPCAdmin"
```

```
POLICY_ARN="arn:aws:iam::aws:policy/AmazonVPCFullAccess"
```

```
echo "====="
```

```
echo " IAM VPC User Creation"
```

```
echo "====="
```

```
echo "Checking AWS identity..."
```

```
aws sts get-caller-identity
```

```
if [ $? -ne 0 ]; then
```

```
    echo "ERROR: AWS CLI authentication failed."
```

```
    exit 1
```

IAM TASK

```
fi

echo

echo "Creating IAM user: $USER_NAME"

aws iam create-user \

    --user-name "$USER_NAME"

if [ $? -ne 0 ]; then

    echo "ERROR: Failed to create IAM user."

    exit 1

fi

echo

echo "Attaching AmazonVPCFullAccess..."

aws iam attach-user-policy \

    --user-name "$USER_NAME" \

    --policy-arn "$POLICY_ARN"

if [ $? -ne 0 ]; then

    echo "ERROR: Failed to attach policy."

    exit 1

fi

echo

echo "====="

echo " User created successfully"

echo "====="

echo "Username:"

echo "$USER_NAME"

echo
```

IAM TASK

```
echo "Attached policies:"
```

```
aws iam list-attached-user-policies \  
  
--user-name "$USER_NAME"
```

Step-8: Make Executable file

```
chmod +x create-vpc-user.sh
```

Step-9: Run Script

```
./create-vpc-user.sh
```

Step-10: Verify User

```
aws iam get-user --user-name VPCAdmin
```

```
aws iam list-attached-user-policies \  
--user-name VPCAdmin
```

```
HP@Ashish MINGW64 ~  
$ aws sts get-caller-identity  
{  
  "UserId": "567487919764",  
  "Account": "567487919764",  
  "Arn": "arn:aws:iam::567487919764:root"  
}
```

Retrieve access key [Info](#)

Access key

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key

Secret access key

 AKIAYIIHKO2KDGAYEGNL

 ***** [Show](#)

IAM TASK

```
HP@Ashish MINGW64 ~
$ aws --version
aws-cli/2.36.32 Python/3.14.6 Windows/11 script-exe/AMD64

HP@Ashish MINGW64 ~
$ aws sts get-caller-identity

aws: [ERROR]: An error occurred (InvalidClientTokenId) when calling the GetCallerIdentity operation: The security token included in the request is invalid.

HP@Ashish MINGW64 ~
$ aws configure
AWS Access Key ID [*****7KWH]: AKIAYIIHKO2KDGAYEGNL
AWS Secret Access Key [*****9ZA4]: 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
Default region name [us-east-1]: us-east-1
Default output format [json]: json

HP@Ashish MINGW64 ~
$ aws sts get-caller-identity
{
  "UserId": "567487919764",
  "Account": "567487919764",
  "Arn": "arn:aws:iam::567487919764:root"
}

HP@Ashish MINGW64 ~
$ aws configure get region
us-east-1

HP@Ashish MINGW64 ~
$ $ aws --version

aws-cli/2.36.32 Python/3.14.6 Windows/11 script-exe/AMD64

HP@Ashish MINGW64 ~
$ aws sts get-caller-identity
```

```
HP@Ashish MINGW64 ~
$ aws configure get region
us-east-1

HP@Ashish MINGW64 ~
$ aws configure set region us-east-1

HP@Ashish MINGW64 ~
$ aws configure get region
us-east-1
```

```
HP@Ashish MINGW64 ~
$ vim create-vpc-user.sh

HP@Ashish MINGW64 ~
$ chmod +x create-vpc-user.sh

HP@Ashish MINGW64 ~
$ ls -l create-vpc-user.sh
-rwxr-xr-x 1 HP 197121 1026 Sep  2 15:22 create-vpc-user.sh*
```

IAM TASK

```
#!/bin/bash

USER_NAME="VPCAdmin"

POLICY_ARN="arn:aws:iam::aws:policy/AmazonVPCFullAccess"

echo "====="
echo " IAM VPC User Creation"
echo "====="

echo "Checking AWS identity..."

aws sts get-caller-identity

if [ $? -ne 0 ]; then
    echo "ERROR: AWS CLI authentication failed."
    exit 1
fi

echo
echo "Creating IAM user: $USER_NAME"

aws iam create-user \
    --user-name "$USER_NAME"

if [ $? -ne 0 ]; then
    echo "ERROR: Failed to create IAM user."
    exit 1
fi

echo
echo "Attaching AmazonVPCFullAccess..."

aws iam attach-user-policy \
    --user-name "$USER_NAME" \
    --policy-arn "$POLICY_ARN"

if [ $? -ne 0 ]; then
    echo "ERROR: Failed to attach policy."
    exit 1
fi

echo
echo "====="
echo " User created successfully"
echo "====="
```

IAM TASK

```
HP@Ashish MINGW64 ~
$ ./create-vpc-user.sh
=====
IAM VPC User Creation
=====
Checking AWS identity...
{
  "UserId": "567487919764",
  "Account": "567487919764",
  "Arn": "arn:aws:iam::567487919764:root"
}

Creating IAM user: VPCAdmin
{
  "User": {
    "Path": "/",
    "UserName": "VPCAdmin",
    "UserId": "AIDAYIIHK02KJDKUYN4SP",
    "Arn": "arn:aws:iam::567487919764:user/VPCAdmin",
    "CreateDate": "2026-09-02T09:53:18+00:00"
  }
}

Attaching AmazonVPCFullAccess...

=====
User created successfully
=====
Username:
VPCAdmin

Attached policies:
{
  "AttachedPolicies": [
    {
      "PolicyName": "AmazonVPCFullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonVPCFullAccess"
    }
  ]
}
```

IAM TASK

```
HP@Ashish MINGW64 ~
$ aws iam get-user --user-name VPCAdmin
{
  "User": {
    "Path": "/",
    "UserName": "VPCAdmin",
    "UserId": "AIDAYIIHK02KJDKUYN4SP",
    "Arn": "arn:aws:iam::567487919764:user/VPCAdmin",
    "CreateDate": "2026-09-02T09:53:18+00:00"
  }
}

HP@Ashish MINGW64 ~
$ aws iam list-attached-user-policies \
> --user-name VPCAdmin
{
  "AttachedPolicies": [
    {
      "PolicyName": "AmazonVPCFullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonVPCFullAccess"
    }
  ]
}
```

VPCAdmin Info

[Delete](#)

Summary

ARN
arn:aws:iam::567487919764:user/VPCAdmin

Created
September 02, 2026, 15:23 (UTC+05:30)

Console access
Disabled

Last console sign-in
-

Access key 1
[Create access key](#)

Permissions
Groups
Tags
Security credentials
Last Accessed

Permissions policies (1)

Permissions are defined by policies attached to the user directly or through groups.

[Simulate](#)
[Remove](#)
[Add permissions](#)

	Policy name	Type	Attached via
☐	AmazonVPCFullAccess	AWS managed	Directly

IAM TASK

5. Create an IAM policy to allow EC2 access for a specific user in specific regions only.

Requirement

User: RegionalEC2User

should have EC2 access only in:

us-east-1

us-west-2

and EC2 operations should not be allowed in:

ap-south-1

eu-west-1

etc.

Step 1 — Create Policy

IAM → Policies → Create policy Create policy

Select: JSON

Enter Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowEC2InApprovedRegions",
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": [
            "us-east-1",
            "us-west-2"
          ]
        }
      }
    },
    {
      "Sid": "DenyEC2OutsideApprovedRegions",
      "Effect": "Deny",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": [
            "us-east-1",
            "us-west-2"
          ]
        }
      }
    }
  ]
}
```

IAM TASK

```
}  
  }  
}  
  }  
}  
}
```

Name Policy: EC2-Specific-Regions-Only

Description: Allows EC2 access only in us-east-1 and us-west-2. --> Create policy

Attach Policy to User: IAM → Users → RegionalEC2User → Permissions → Add permissions

Select: Attach policies directly

Find: EC2-Specific-Regions-Only

Test

Set your CLI region:

aws configure set region us-east-1

Check:

aws ec2 describe-instances

This should be permitted, assuming the user has the required authentication and no other policy denies it.

Now:

aws configure set region ap-south-1

Run:

aws ec2 describe-instances

With the explicit Deny policy, the request should be denied.

Review and create [Info](#)

Review the permissions, specify details, and tags.

Policy details

Policy name

Enter a meaningful name to identify this policy.

EC2-Specific-Regions-Only

Maximum 128 characters. Use alphanumeric and '+','=','@','_','.' characters.

Description - *optional*

Add a short explanation for this policy.

Allows EC2 access only in us-east-1 and us-west-2

Maximum 1,000 characters. Use alphanumeric and '+','=','@','_','.' characters.

Permissions defined in this policy [Info](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it (1)

Q Search

Explicit deny (1 of 475 services)

IAM TASK

Policy editor

Visual

1 * {
2 "Version": "2012-10-17",
3 "Statement": [
4 {
5 "Sid": "AllowEC2InApprovedRegions",
6 "Effect": "Allow",
7 "Action": "ec2:*",
8 "Resource": "*",
9 "Condition": {
10 "StringEquals": {
11 "aws:RequestedRegion": [
12 "us-east-1",
13 "us-west-2"
14]
15 }
16 }
17 }
18]
19 }
20 "Sid": "DenyEC2OutsideApprovedRegions",
21 "Effect": "Deny",
22 "Action": "ec2:*",
23 "Resource": "*",
24 "Condition": {
25 "StringNotEquals": {
26 "aws:RequestedRegion": [
27 "us-east-1",
28 "us-west-2"
29]
30 }
31 }
32]
33 }

Edit

Allo

Add

Cho

Q

Incl

EC2

Ava

AI C

AMI

API

API

ARC

ARC

ASC

Acco

Add

Policy EC2-Specific-Regions-Only created.

View policy

×

EC2-Specific-Regions-Only

Info

Edit

Delete

Allows EC2 access only in us-east-1 and us-west-2.

Policy details

Type	Creation time	Edited time	ARN
Customer managed	September 02, 2026, 16:00 (UTC+05:30)	September 02, 2026, 16:00 (UTC+05:30)	arn:aws:iam::567487919764:policy/EC2-Specific-Regions-Only

Permissions

Entities attached

Tags

Policy versions (1)

Last Accessed

Permissions defined in this policy

Info

Edit

Summary

JSON

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

(1)

Q Search

Explicit deny (1 of 475 services)

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐ Add user to group
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions
Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒ Attach policies directly
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1/1572)

Create policy

Choose one or more policies to attach to your new user.

Q EC2-Specific-Regions-Only

Filter by Type

All types

1 match

< 1 >

Policy name	Type	Attached entities
☒ EC2-Specific-Regions-Only	Customer managed	0

IAM TASK

```
HP@Ashish MINGW64 ~
$ aws configure set region us-east-1

HP@Ashish MINGW64 ~
$ aws ec2 describe-instances
{
  "Reservations": []
}

HP@Ashish MINGW64 ~
$ aws configure set region ap-south-1

HP@Ashish MINGW64 ~
$ aws ec2 describe-instances
{
  "Reservations": []
}

HP@Ashish MINGW64 ~
$ |
```

6. We have two accounts: Account A and Account B. Account A user should access an S3 bucket in Account B.

(Collaborate with a team member and execute this. This is mostly asked in every interview.)

AWS Cross-Account S3 Access

Objective

Configure cross-account access so that an IAM user in AWS Account A can access an S3 bucket owned by AWS Account B.

Account Details

Account	Account ID	Resource
Account A	108457943052	IAM User: S3-swap
Account B	567487919764	S3 Bucket: task654321
Bucket ARN	—	arn:aws:s3:::task654321

Steps Performed

Step 1: Created the S3 Bucket

In Account B (567487919764), created the S3 bucket:
task654321

IAM TASK

Bucket ARN: arn:aws:s3:::task654321

Step 2: Created/Used IAM User in Account A

In Account A (108457943052), used the IAM user:

S3-swap

Created access keys for the IAM user so that the AWS CLI could authenticate as S3-swap.

Step 3: Configured AWS CLI Profile

Initially, the AWS CLI was using the default profile, which was authenticated as the Account A root user.

Verified the identity using:

```
aws sts get-caller-identity
```

After creating a dedicated profile for S3-swap, configured it using:

```
aws configure --profile S3-swap
```

Verified the profile:

```
aws sts get-caller-identity --profile S3-swap
```

The identity was confirmed as:

```
arn:aws:iam::108457943052:user/S3-swap
```

Step 4: Added IAM Permissions in Account A

Granted S3-swap permission to list the bucket and perform object operations.

The permissions included:

```
s3:ListBucket
```

```
s3:GetObject
```

```
s3:PutObject
```

```
s3:DeleteObject
```

The bucket-level resource was:

```
arn:aws:s3:::task654321
```

The object-level resource was:

```
arn:aws:s3:::task654321/*
```

Step 5: Added Bucket Policy in Account B

In Account B, added a bucket policy to trust the IAM user from Account A.

The principal was:

```
arn:aws:iam::108457943052:user/S3-swap
```

The bucket policy allowed the required S3 operations on:

```
arn:aws:s3:::task654321
```

and:

```
arn:aws:s3:::task654321/*
```

IAM TASK

Step 6: Tested Cross-Account Access

Used the S3-swap CLI profile to access the bucket:

```
aws s3 ls s3://task654321 --profile S3-swap
```

Then tested uploading an object:

```
echo "Hello from Account A" > test.txt
```

```
aws s3 cp test.txt s3://task654321/test.txt --profile S3-swap
```

The upload was successful.

The object could also be listed/downloaded using the S3-swap profile.

Initial Issue and Resolution

Initially, the AWS CLI was authenticated as:

```
arn:aws:iam::108457943052:root
```

Therefore, the bucket policy allowing S3-swap did not apply to the request.

A dedicated AWS CLI profile was created:

S3-swap

After switching to this profile, the request was correctly authenticated as:

```
arn:aws:iam::108457943052:user/S3-swap
```

Cross-account S3 access then worked successfully.

Final Architecture

AWS Account A

Account ID: 108457943052 | IAM User | S3-swap | Cross-account S3 request

AWS Account B

Account ID: 567487919764 | S3 Bucket-task654321-- test.txt

Conclusion

Cross-account access between AWS Account A and AWS Account B was successfully configured.

The IAM user S3-swap in Account A (108457943052) was granted access to the S3 bucket task654321 in Account B (567487919764) through a combination of:

1. IAM permissions attached to S3-swap in Account A.
2. S3 bucket policy in Account B that trusts S3-swap.
3. AWS CLI profile configured with the credentials of S3-swap.

The configuration was successfully validated by uploading an object from Account A to the S3 bucket in Account B.

IAM TASK

This demonstrates how AWS supports cross-account resource access using IAM policies and resource-based policies.

```
S3-swap
Srees-MacBook-Air:~ sreevemuri$ aws sts get-caller-identity --profile S3-swap
{
  "UserId": "AIDARSQEY4AGBZLK4W5LR",
  "Account": "108457943052",
  "Arn": "arn:aws:iam::108457943052:user/S3-swap"
}
Srees-MacBook-Air:~ sreevemuri$ aws s3 ls s3://task654321 --profile S3-swap

aws: [ERROR]: An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied
Srees-MacBook-Air:~ sreevemuri$ aws s3 ls s3://task654321 --profile S3-swap
Srees-MacBook-Air:~ sreevemuri$ ws s3 cp test.txt s3://task654321/test.txt --profile S3-swap
-bash: ws: command not found
Srees-MacBook-Air:~ sreevemuri$ ws s3 cp test.txt s3://task654321/test.txt --profile S3-swap
-bash: ws: command not found
Srees-MacBook-Air:~ sreevemuri$ aws s3 cp test.txt s3://task654321/test.txt --profile S3-swap
upload: ./test.txt to s3://task654321/test.txt
```

Permissions defined in this policy [Info](#)

[Copy](#)[Edit](#)[Summary](#)[JSON](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "s3:ListBucket",
7       "Resource": "arn:aws:s3:::task654321"
8     },
9     {
10      "Effect": "Allow",
11      "Action": [
12        "s3:GetObject",
13        "s3:PutObject",
14        "s3:DeleteObject"
15      ],
16      "Resource": "arn:aws:s3:::task654321/*"
17    }
18  ]
19 }
```

```
Default region name [None]: us-east-1
Default output format [None]: json
[Srees-MacBook-Air:~ sreevemuri$ aws configure list-profiles
default
S3-swap
Srees-MacBook-Air:~ sreevemuri$ aws sts get-caller-identity --profile S3-swap
{
  "UserId": "AIDARSQEY4AGBZLK4W5LR",
  "Account": "108457943052",
  "Arn": "arn:aws:iam::108457943052:user/S3-swap"
}
[Srees-MacBook-Air:~ sreevemuri$ aws s3 ls s3://task654321 --profile S3-swap

aws: [ERROR]: An error occurred (AccessDenied) when calling the ListObjectsV2 operation: Access Denied
[Srees-MacBook-Air:~ sreevemuri$ aws s3 ls s3://task654321 --profile S3-swap
Srees-MacBook-Air:~ sreevemuri$ ws s3 cp test.txt s3://task654321/test.txt --profile S3-swap
-bash: ws: command not found
[Srees-MacBook-Air:~ sreevemuri$ ws s3 cp test.txt s3://task654321/test.txt --profile S3-swap
-bash: ws: command not found
[Srees-MacBook-Air:~ sreevemuri$ aws s3 cp test.txt s3://task654321/test.txt --profile S3-swap
upload: ./test.txt to s3://task654321/test.txt
[Srees-MacBook-Air:~ sreevemuri$ aws s3 cp s3://task654321/test.txt downloaded.txt --profile S3-swap
download: s3://task654321/test.txt to ./downloaded.txt
```

IAM TASK

Amazon S3

Buckets

task654321

Bucket policy

EditDelete

Public access is blocked because Block Public Access settings are turned on for this bucket

To determine which settings are turned on, check your Block Public Access settings for this bucket. Learn more about using Amazon S3 Block Public Access

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3SwapList",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::108457943052:user/S3-swap"
      },
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::task654321"
    },
    {
      "Sid": "AllowS3SwapObjects",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::108457943052:user/S3-swap"
      },
      "Action": [
        "s3:GetObject",

```

Copy

User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

View user

DevopsA

Info

Delete

Summary

ARN
arn:aws:iam::567487919764:user/DevopsA

Console access
Disabled

Access key 1
Create access key

Created
September 02, 2026, 17:25 (UTC+05:30)

Last console sign-in
-

Permissions

Groups

Tags

Security credentials

Last Accessed

Permissions policies (0)

Simulate iRemoveAdd permissions

Permissions are defined by policies attached to the user directly or through groups.

Search

Filter by Type
All types

< 1 >

Policy nameTypeAttached via